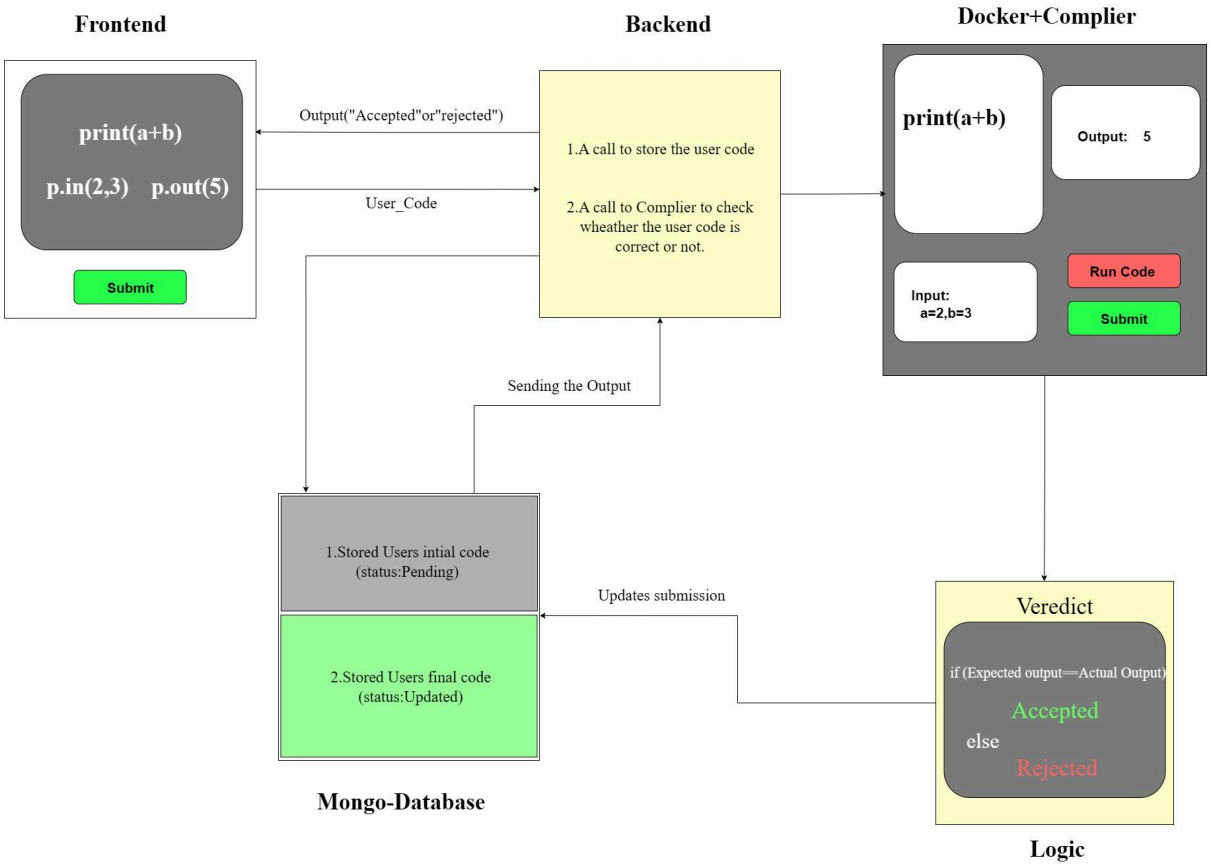
Online Judge Platform

The Online Judge Platform is a full-stack web application designed to allow users to browse coding problems, write and execute code, and receive evaluation verdicts. Its architecture separates concerns into several key components to ensure scalability, security, and efficient code execution.

Overall System Architecture:

The core flow involves a user interacting with the Frontend, which communicates with the Backend API. The Backend then manages submissions, pushing them to a Judge Worker System for execution within isolated Docker containers. All persistent data is managed by MongoDB.



Component-wise Breakdown:

1. **Frontend (Client)**
    - **Tech Stack:** React.js, Tailwind CSS, Axios

🔹 React.js (Frontend Framework)

We used React.js to build a fast and interactive user interface. It allows us to efficiently update and manage UI components without reloading the page. Compared to traditional HTML or jQuery, React makes the app more responsive and easier to scale with modern frontend architecture.

🔹 Tailwind CSS (Styling Framework)

Tailwind CSS helps us create clean, responsive designs using utility classes directly in HTML. It is faster and more flexible than traditional CSS or frameworks like Bootstrap because we don't need to write separate CSS files for every component. This saved development time and made UI consistency easier.

🔹 Axios (HTTP Client)

We used **Axios** to send and receive data between the frontend and backend. It simplifies API requests with easy syntax and better error handling than the built-in `fetch` API. Axios also supports features like interceptors, which are helpful for handling tokens (like JWT)in secure requests.

    - **Responsibilities:**
        - User authentication (login/register)
        - Displaying problem lists with filtering capabilities
        - Presenting detailed problem descriptions
        - Providing an interactive code editor (Monaco/CodeMirror)
        - Allowing users to select programming languages
        - Displaying submission history and results
    - **Interaction:** Communicates with the Backend API for data retrieval and submission.

2. **Backend (API Server)**
    - **Tech Stack:** Node.js (Express)

🔹 Node.js with Express.js (Backend Server)

Node.js with **Express.js** powers the backend and handles API requests, user authentication, and code submissions. It supports asynchronous operations, which makes it perfect for handling multiple code submissions at once. We chose it over languages like PHP or Java because it's lightweight, fast, and works well with JavaScript across the full stack.

- **Responsibilities:**
    - Handling user authentication using JWT (JSON Web Tokens)
    - Serving coding problems to the Frontend
    - Receiving and validating user code submissions
    - Managing a queue for judge submissions (likely using Redis)
    - Returning execution verdicts to the Frontend

- **Interaction:**
    - Receives requests from the Frontend.
    - Interacts with MongoDB for data persistence.
    - Sends submission jobs to the Judge Worker System.

3. **Database (MongoDB)**
    - **Type:** NoSQL Document Database

🔹 MongoDB (NoSQL Database)

MongoDB is a flexible NoSQL database that stores user data, problems, test cases, and submissions as documents. It's easier to work with than SQL databases for dynamic data structures and scales well with large data sets. Its JSON-like format also integrates naturally with JavaScript-based applications.

- **Collections:**
  - `users`: Stores `user_id,password,user_name`, `email`, and `password_hash`.
  - `problems`: Stores `problem_id,title`, `description`, and `input,output,constraints` for each coding problem.
  - `test_cases`: Stores `problem_id`, `input`, and `expected_output` for problem validation.
  - `submissions`: Stores `problem_id,user_id`, submitted `code`, the `verdict`, and `runtime` of submissions.
- **Interaction:** Accessed by the Backend API and updated by the Judge System.

3. **Judge System (Complier)+Logic**
   - **Tech Stack:** Node.js(Express.js)

🔹 Redis (Job Queue - Optional/Future)

We plan to use Redis as a queue system to manage code submissions. It is extremely fast and reliable for storing tasks temporarily. Compared to databases, it handles real-time jobs more efficiently and is widely used in production systems for managing background tasks.

- **Process Flow:**
  - Fetches a submission from the job queue (likely Redis).
  - Executes the user's code within an isolated Docker container.
  - Compares the output of the executed code against the expected output from `test_cases`.
  - Updates the `submissions` collection in MongoDB with the `verdict` and `runtime`.
- **Interaction:**
  - Receives jobs from the Backend via a queue.
  - Spawns and manages Docker containers for code execution.
  - Updates MongoDB with results.

4. **Docker (Code Execution Environment)**
    - **Purpose:** Provides a secure and isolated environment for running submitted code.

🔹 Docker (Code Execution Environment)

Docker provides a secure, isolated environment for running user-submitted code. Each user's code runs in a separate container with limited resources to prevent abuse. We chose Docker over traditional VMs because it is lightweight, faster to start, and easier to manage at scale.

- **Key Features:**
    - Each submission runs in its own dedicated container.
    - Resource limits (CPU, memory) are imposed to prevent abuse.
    - Strict security measures are in place: no network access and no filesystem access from within the containers.

Data Flow (Sample Flow):

1. A user submits their code via the Frontend.
2. The Backend API receives the submission and queues it for judging.
3. The Judge Worker System picks up the job from the queue.
4. The Judge Worker executes the user's code inside a Docker container, running it against predefined test cases.
5. The result (e.g., Accepted, Rejected) is saved to the MongoDB database.
6. The Frontend then retrieves and displays the verdict to the user.

Note:

Few features are added for future upgrading purpose like Queue part